

Scaling Self-Play for End-to-End Driving

Luke Rowe^{1,2}, Roger Girgis^{1,3,4}, Rodrigue de Schaetzen^{1,2,4},
Daphne Cornelisse⁵, Alaap Grandhi^{1,6}, Felix Heide^{4,7}, Eugene Vinitzky⁵,
Christopher Pal^{1,2,3}, Liam Paull^{1,2}

¹Mila, ²Université de Montréal, ³Polytechnique Montréal, ⁴Torc Robotics,
⁵NYU Tandon School of Engineering, ⁶McMaster University, ⁷Princeton University
<https://montrealrobotics.ca/gigapixel>

Abstract: End-to-end autonomous driving models are typically trained on offline human-demonstration datasets that provide limited state coverage and often no closed-loop feedback, making them prone to compounding errors when deployed in closed-loop and brittle to long-tail agent interactions. To overcome these limitations, we propose an alternative strategy for training end-to-end driving models: large-scale self-play directly from pixels in simulation. While prior self-play approaches have shown promising transfer to real-world driving, they typically assume vectorized Bird’s-Eye-View (BEV) observations that are incompatible with end-to-end policies operating directly on sensor observations. To this end, we introduce *Gigapixel*, a high-throughput batched driving simulator with perspective rendering, enabling scalable self-play directly from pixel observations. Rather than targeting compute-costly photorealistic sensor simulation, Gigapixel renders a simplified bounding-box world that preserves essential scene structure while achieving throughput at 50k agent steps per second. Since direct pixel-space self-play RL is prohibitively sample-inefficient at end-to-end model scale, we propose *self-play DAGger* training: we train pixel-based policies in self-play via on-policy distillation from a privileged RL teacher. To bridge the sim-to-real gap, we subsequently transfer the self-play trained policies to real-world sensor data through lightweight perception adaptation. Policies trained in Gigapixel and adapted to real-world sensor data achieve competitive performance on the HUGSIM and NAVSIM-v2 benchmarks without human trajectory supervision. Moreover, scaling self-play training yields proportional gains in policy performance, establishing self-play as a practical and scalable strategy for training end-to-end models.

Keywords: End-to-End Autonomous Driving, Self-Play, Simulation

1 Introduction

Autonomous driving has undergone a decisive shift towards end-to-end models that directly map sensor inputs to planning outputs [1, 2, 3, 4, 5, 6, 7]. Beyond architectural simplicity, end-to-end formulations are inherently scalable: they optimize the planning objective directly, and performance improves predictably with increasing data and model capacity [8, 9, 10]. Realizing this scaling potential, however, requires a principled and scalable training approach. Behavior cloning (BC) of human driving logs remains the dominant training paradigm [11, 8, 9, 10, 7], but it suffers from structural limitations. Logged datasets provide limited state coverage, leaving policies brittle once they reach states outside nominal human driving [12]. Moreover, the absence of closed-loop interaction during training induces covariate shift, leading to compounding errors at deployment [13]. These shortcomings are intrinsic to BC and do not disappear merely from collecting more data [14, 15].

Self-play in simulation offers a principled alternative. Agents learn through *closed-loop* interaction with copies of themselves, generating a diverse experience curriculum during training. Closed-loop interaction enables learning the consequences of its own actions, which improves robustness to compounding errors at test time [13]. Furthermore, unlike offline datasets, which are fixed and

costly to expand, self-play state coverage scales directly with compute, enabling arbitrarily large and targeted experience generation. Realizing this benefit at scale requires a high-throughput simulator to cheaply generate on-policy experience. Recent batched simulators such as Gigaflow [16] and PufferDrive [17] demonstrate throughputs exceeding hundreds of thousands of steps per second (SPS), enabling robust closed-loop learning from self-play [18, 19, 20, 16, 17].

However, these simulators produce vectorized BEV observations incompatible with end-to-end policies that must act from raw sensor inputs. To overcome this limitation, we introduce *Gigapixel*, a high-throughput batched driving simulator that extends PufferDrive with ray-traced and rasterized perspective rendering [22], enabling **scalable self-play training directly from pixel observations at 50k agent SPS on 1 GPU**, with throughput scaling near-linearly in the number of GPUs. Rather than simulating photorealistic sensors, Gigapixel renders perspective views of a simplified bounding-box world that preserves essential scene geometry and interaction fidelity.

While Gigapixel enables high-throughput batched simulation from pixels, direct self-play RL at end-to-end model scale remains prohibitively sample-inefficient due to the cost of policy forward and backward passes (Figure 1). As a more sample-efficient alternative, we introduce *self-play DAGger*: a privileged teacher policy is first trained via RL on vectorized observations, then distilled into a pixel-based student via DAGger [14, 23] in self-play. This preserves the benefits of closed-loop self-play while dramatically reducing sample complexity. Self-play DAGger additionally enables training a trajectory-output policy, matching the output format of standard end-to-end planners [4, 5, 24, 25], rather than the control outputs of typical RL-based planners [26, 16]. Then, to deploy these policies on real sensor data, we isolate the sim-to-real gap to perception: the planning head already captures the closed-loop behaviors learned in self-play, so we adapt only the perception module to map real sensor inputs into the planning head’s latent representation.

We evaluate Gigapixel self-play trained policies in simulation and on real-world benchmarks. Our approach achieves state-of-the-art performance on the closed-loop HUGSIM benchmark [27] and competitive performance on the pseudo-closed-loop NAVSIM-v2 benchmark [28], *without any human trajectory supervision*. Empirically, self-play training scales consistently: increasing training experience yields proportional gains in policy performance. We summarize our contributions: **1.** We propose self-play as a scalable alternative to offline behavior cloning for training end-to-end driving models. We enable this through self-play DAGger, a sample-efficient alternative to direct self-play RL from pixels. **2.** We introduce Gigapixel, a high-throughput batched perspective-rendering simulator that enables pixel-based self-play at scale. **3.** We show that the robust driving behavior learned via self-play in simulation transfers to robust driving from real-world observations through lightweight sim-to-real perception adaptation.

Together, these contributions establish self-play as a scalable and practical strategy for training end-to-end driving models—one that addresses the structural limitations of offline behavior cloning and opens a path to continual improvement through synthetic experience.

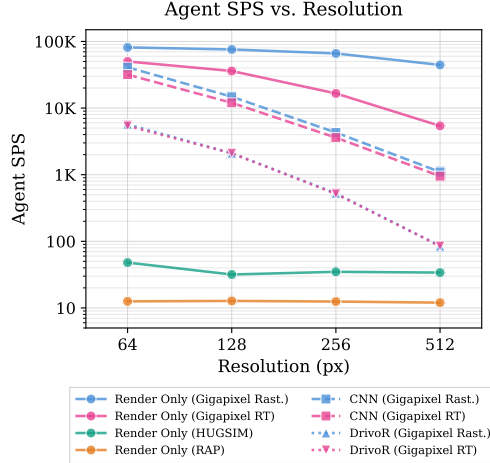


Figure 1: **Gigapixel Throughput vs. Resolution.** Agent steps per second (SPS) across rendering resolutions and policy architectures on 1 NVIDIA A100L GPU. *Render Only* isolates renderer throughput without policy forward or backward passes. *CNN* is a simple CNN, and *DrivoR* [21] is a transformer-based architecture. CNN and DrivoR throughputs are reported in an RL training loop. The gap between the rasterizer (*Rast.*) and ray-tracer (*RT*) throughput narrows as model complexity increases, indicating that Gigapixel rendering ceases to be the throughput bottleneck. For comparison, we report HUGSIM and RAP throughputs using their public code.

2 Related work

End-to-End Autonomous Driving. The growing availability of large-scale datasets of human driving logs [29, 30, 31, 32, 33, 34] have enabled tremendous progress in behavior cloning for end-to-end driving [11], spanning transformer-based [4, 5, 3, 35, 36, 21], diffusion-based [25, 37], and vision-language model-based [6, 38, 39, 7] methods. Despite this progress, behavior cloning suffers from structural limitations that induce brittle behavior when deployed in closed-loop [13, 15, 14, 40, 41]. To address these limitations, prior works have explored direct closed-loop training in simulators via RL either from scratch [42, 43, 44, 45, 46] or as a post-training stage [47, 48]. Other works propose DAgger-based methods [14, 49] with privileged state-based experts [50, 51, 52], test-time adaptation [53], or rollouts in a latent world model [54]. However, these prior methods predominantly focus on the single-agent paradigm and therefore train on environments with limited behavioral diversity. Moreover, prior works typically train in slow simulators such as CARLA [55, 56, 44, 57] or neural reconstruction-based simulators [27, 48] that limit scalability. Our method is a closed-loop DAgger-based method, but unlike prior works, we propose a *multi-agent* training paradigm that leverages the benefits of scalable self-play in a fast simulator [16] to learn a robust end-to-end driving policy.

Autonomous Driving Simulators. The space of driving simulators is characterized by a tension between throughput and representational fidelity. At one extreme, abstract simulators operate on vectorized BEV state representations [30, 58, 17, 59, 60], sacrificing perceptual realism for speed. At the other end, photorealistic simulators render high-fidelity sensor data to minimize the sim-to-real gap, either via handcrafted assets [61, 55, 62], neural reconstruction methods [63, 64, 27, 65], or generative models [66, 67, 68, 69, 70]. However, this comes at a dramatic cost of training throughput (steps per second or SPS). We argue that Gigapixel occupies a productive middle ground: it retains the throughput necessary to experiment with self-play and closed-loop training at scale while providing perspective-view observations that make it suitable for training end-to-end driving policies. The representational approach that most closely mirrors Gigapixel is RAP [71], which also renders simplified bounding box worlds. However, as shown in Figure 1, RAP rendering is significantly slower than Gigapixel due to the lack of batched GPU rendering support [72, 22], and RAP utilizes these simplified renderings for data augmentation rather than closed-loop training.

Self-Play for Driving. Self-play RL algorithms are data-hungry, requiring billions of steps of experience to reach (super) human-level performance. The appeal of self-play lies in the nature of the experience it generates. Unlike behavior cloning, which learns from a fixed and narrow distribution of human demonstrations, self-play agents are paired with themselves and are given an objective to optimize. The distribution of encountered behaviors continually shifts as they learn, surfacing many safety-critical interactions that are vanishingly rare in human driving logs [73, 20, 16]. The effectiveness of this paradigm has its roots in competitive and cooperative games [74, 75, 76, 77]. Thus far, the results in driving have been confined to policies operating on vectorized observations [18, 19, 78, 16, 79, 80, 81]. We go beyond prior works by enabling large-scale self-play directly from pixel observations. Moreover, we propose a closed-loop self-play training procedure based on DAgger rather than RL to enable sample-efficient self-play learning at end-to-end model scale.

3 Self-Play for End-to-End Driving

Problem formulation. Our goal is to learn a robust end-to-end driving policy $\pi(\tau|I, C)$ that outputs a trajectory τ from raw image observations I and additional ego context C (e.g., ego state and navigation command). We represent a trajectory as a sequence of waypoints $\tau = \{(x_t, y_t, \theta_t)\}_{t=1}^H$, where (x_t, y_t) denotes the ego-centric position and θ_t the heading of the ego vehicle at future timestep t , over a finite planning horizon H . A low-level controller maps τ to a sequence of control actions, of which the first action a is applied before replanning at the next timestep in a closed-loop (receding-horizon) fashion. We assume π decomposes into a perception backbone $f_{\theta_{\text{per}}}$ that produces perception features $E_{\text{per}} = f_{\theta_{\text{per}}}(I, C)$, and a planning head $f_{\theta_{\text{plan}}}$ that maps these features to a

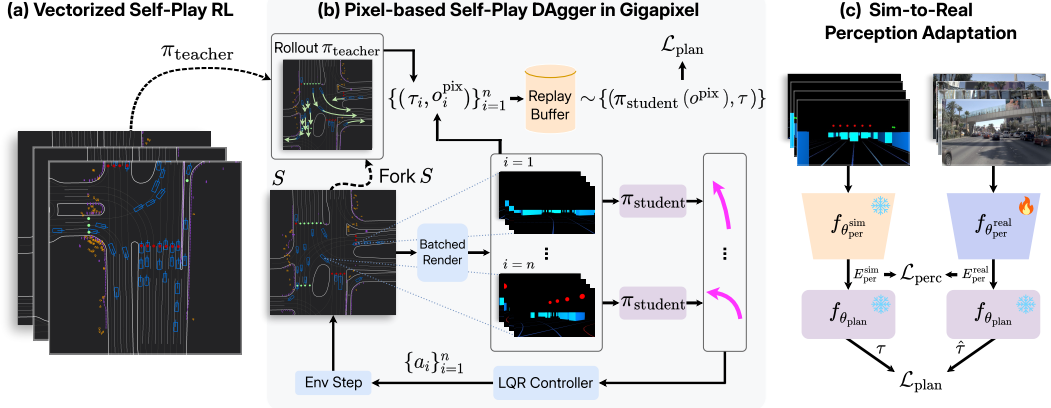


Figure 2: **Self-Play for End-to-End Driving.** We propose self-play training for end-to-end driving in three stages: **(a)** a vectorized teacher is trained with self-play RL in an abstract simulator; **(b)** a pixel-based student is distilled via self-play DAgger in Gigapixel, with all agents student-controlled and trajectory targets generated by parallel teacher rollouts (Sec. 3.2); and **(c)** the student is adapted to real images by finetuning only the perception backbone on paired sim-real observations (Sec. 3.3).

trajectory, $\tau = f_{\theta_{\text{plan}}}(E_{\text{per}}, C)$. This decomposition is general and satisfied by a wide range of end-to-end architectures [3, 21, 4, 5, 24, 25, 38]. We place no constraints on $f_{\theta_{\text{plan}}}$ beyond its input-output interface; it may be scoring-based [36, 21], diffusion-based [25], or regression-based [3, 38].

Online RL and DAgger. Self-play is a *closed-loop* training paradigm: the agent rolls out actions in the simulator, allowing it to learn the consequences of its own actions during training [13]. Two standard paradigms enable closed-loop learning in simulation. Online RL [82, 83, 84, 85, 86] trains a policy π to maximize the expected discounted return $\mathbb{E}_{\pi}[\sum_t \gamma^t r_t]$ with discount factor γ , where r_t is the reward obtained at step t . The policy is rolled out in the simulator and trained on the generated experience. While effective, online RL is sample-inefficient [82], often requiring billions of environment steps to converge. This becomes prohibitive when the policy is a large end-to-end model, as each step incurs costly forward and backward passes through the model. Dataset Aggregation (DAgger) [14] offers a more sample-efficient alternative when an expert policy is available. At each iteration, the student policy π_{student} is rolled out in the environment, the visited states are labeled by an expert π_{expert} , and the student is trained to match the expert’s actions on the student’s own induced state distribution. Crucially, the student and expert may each act through its own observation function, denoted by O_{student} and O_{expert} . The DAgger objective is:

$$\min_{\theta} \mathbb{E}_{S \sim d_{\pi_{\text{student}}}} [\mathcal{L}(\pi_{\text{student}}^{\theta}(O_{\text{student}}(S)), \pi_{\text{expert}}(O_{\text{expert}}(S)))], \quad (1)$$

with action loss $\mathcal{L}$. Like RL, DAgger is closed-loop: the expectation is taken under π_{student} ’s own state distribution $d_{\pi_{\text{student}}}$, so the student learns to recover from the states it actually visits.

3.1 Gigapixel

A key obstacle to leveraging self-play for end-to-end driving is that existing simulators either natively expose *vectorized* observations incompatible with end-to-end policies [16, 59, 17, 30, 60], or are too slow to support self-play at scale [55, 27, 69, 30]. Our central observation is that learning robust closed-loop end-to-end driving behavior does not require a photorealistic sensor simulator. Instead, it suffices to learn in a *high-throughput* abstract simulator that preserves the essential information required for planning while natively supporting pixel observations. To this end, we introduce *Gigapixel*, a driving simulator that enables scalable self-play directly from pixel observations. In Gigapixel, the global simulator state takes the form $S_t = (A_t, M, L_t)$, comprising agent bounding boxes A_t , static map polylines M , and traffic light states L_t . Prior abstract simulators implement vectorized observations $O_{\text{vec}}(S_t)$ [17, 16], which are cheap to produce but incompatible with end-to-end policies that consume raw sensor inputs. Gigapixel additionally implements

$O_{\text{pixel}}(S_t)$, rendering S_t into an ego-centric perspective view through a batched, GPU-accelerated renderer. Concretely, we extend the PufferDrive abstract simulator [17] with the Madrona rendering engine [72, 22], supporting both rasterization [87, 71] and ray tracing [88]. To sustain high throughput, scene elements are represented as simple primitives: agents (*e.g.*, vehicles, pedestrians, cyclists) and static obstacles as cuboids, lane polylines as thin planar strips, and traffic lights as small spheres. Figure 2 shows ray-traced renderings; rasterized counterparts appear in the Appendix. Further details about the Gigapixel simulator—including the renderer, dynamics, action space, and reward functions—are deferred to the Appendix.

3.2 Self-play DAgger

Self-play RL from pixels faces two obstacles in our setting. First, training compute-intensive end-to-end architectures in Gigapixel shifts the per-step cost from rendering to policy forward and backward passes; combined with RL’s sample inefficiency, this makes self-play RL from pixels prohibitively expensive. Second, end-to-end policies typically output trajectories τ , whereas RL-trained driving policies typically output low-level controls [89, 90, 26, 16]. To address both, we propose *self-play DAgger* training: pixel-based end-to-end policies are trained in self-play via on-policy distillation from a privileged RL teacher. Critically, RL is tractable for the vectorized teacher but prohibitive for the pixel-based student (Fig. 1): the teacher is a lightweight policy over low-dimensional vectorized observations, making its forward/backward passes cheap, and self-play RL is known to scale to robust, naturalistic driving in high-throughput simulators [16, 19]. We therefore confine RL to the teacher and distill its closed-loop behavior into the expensive pixel-based student. Distillation is far more sample-efficient than online RL (see Fig. 3), and rolling out the teacher in a forked parallel environment produces trajectory targets τ directly, sidestepping the waypoint-vs-controls mismatch.

Self-play DAgger vs. vanilla DAgger. Standard DAgger assumes a single learner whose visited states are labeled by an expert. In autonomous driving, this typically means the learner controls the ego vehicle while other agents follow log-replay or handcrafted behaviors. In our setting, every agent in the scene is controlled by the student policy π_{student} , so the induced state distribution is a function of all agents’ joint behavior. The student is trained on states drawn from the self-play rollout distribution $S \sim d_{\pi_{\text{student}}}^{\text{self-play}}$, the marginal state distribution induced when π_{student} controls all agents. This gives the self-play DAgger objective:

$$\min_{\theta} \mathbb{E}_{S \sim d_{\pi_{\text{student}}}^{\text{self-play}}} \mathbb{E}_i [\mathcal{L}(\pi_{\text{student}}^{\theta}(O_{\text{student}}(S, i)), \pi_{\text{teacher}}(O_{\text{teacher}}(S, i)))] ,$$

which is Eq. 1 with the self-play induced state distribution $d_{\pi_{\text{student}}}^{\text{self-play}}$ in place of the single-learner distribution and an added expectation over agents i . This yields two benefits over vanilla DAgger. First, because all agents are controlled by π_{student} , the scenarios encountered during training co-evolve with the policy, continually surfacing interesting multi-agent interactions that are rare in human driving logs. Second, as made explicit by the inner expectation over i , every agent in a joint rollout contributes training data, multiplying the training experience extracted per simulator step over vanilla DAgger. We now describe our specific instantiation of self-play DAgger.

Vectorized teacher training. Self-play DAgger requires a teacher π_{teacher} that is robust across the state distribution visited by the student and can be queried cheaply for supervision. We obtain our teacher by adapting Gigaflo [16] which generates robust and naturalistic driving policies from self-play RL, albeit over vectorized BEV representations. We train a compact decentralized policy $\pi_{\text{teacher}}(a_i | o_i^{\text{vec}})$ on ego-centric vectorized observations $o_i^{\text{vec}} = O_{\text{vec}}(S, i)$. Training uses decentralized PPO [86] with a linearly weighted multi-objective reward $R_i = \sum_{j=1}^{N_r} c_i^j R^j$ consisting of N_r individual reward terms (such as collision, offroad, comfort, *etc.*). On each episode reset, the coefficients $\mathbf{c}_i := (c_i^1, \dots, c_i^{N_r})$ are randomized per agent i and provided to the policy as conditioning [91], so a single policy models different driving *personas* (*e.g.*, cautious vs. aggressive). This induces a diverse population of agents during self-play training, improving the policy’s robustness.

Method	Human Trajectory?	RC					HD-Score				
		E	M	H	X	Avg.	E	M	H	X	Avg.
LTF [3]	✓	67.8	35.1	26.2	40.5	38.9	58.9	18.0	9.8	25.9	23.7
VAD [5]	✓	51.3	31.1	25.3	26.5	31.4	36.3	9.5	8.0	11.5	13.4
UniAD [4]	✓	78.4	60.5	33.6	17.8	45.9	64.9	45.8	20.6	6.6	32.7
<i>Regression-based (Single-Mode) DrivoR Planner</i>											
DrivoR-Reg [21]	✓	75.8	28.0	32.3	46.8	40.5	60.3	8.0	11.5	25.8	20.7
Gigapixel-DrivoR-Reg	✗	84.1	54.3	29.8	41.1	49.2	65.8	38.8	13.1	24.6	33.2
<i>Scoring-based (Multimodal) DrivoR Planner</i>											
DrivoR [21]	✓	80.9	50.5	33.8	47.1	<u>49.8</u>	73.3	34.6	18.8	32.5	35.7
DrivoR (w/ SimScale) [21]	✓	70.4	54.4	28.2	39.2	46.4	64.8	50.3	16.8	25.8	<u>38.1</u>
Gigapixel-DrivoR	✗	78.6	60.7	32.7	35.4	50.1	67.4	51.9	19.1	21.6	38.5

Table 1: **Closed-loop evaluation in the photorealistic HUGSIM simulator [27].** Scores are reported per difficulty level (E=Easy, M=Medium, H=Hard, X=Extreme). RC=Route Completion, and HD-Score is the HUGSIM Driving Score. *Human Trajectory?* denotes whether the human trajectory is used as a supervision target during training. Best method is **bolded**; second-best underlined.

Pixel-based student training. Given a trained teacher π_{teacher} , we distill it into the pixel-based student π_{student} via self-play DAgger, where π_{teacher} provides *trajectory-level* supervision. Every agent i is controlled by π_{student} , which acts on the ego-centric pixel observation $o_i^{\text{pix}} = O_{\text{pix}}(S, i)$ and outputs a trajectory tracked by an LQR controller; the resulting joint behavior induces the global simulator state S . To generate supervision at S , we fork a parallel simulator instance initialized at S and roll out the teacher in self-play for H steps, acting on o_i^{vec} . This yields a per-agent trajectory target τ_i for every agent i in the scene, giving us the self-play DAgger objective:

$$\min_{\theta} \mathbb{E}_{S \sim d_{\pi_{\text{student}}^{\text{self-play}}}} \mathbb{E}_i \left[\mathcal{L}_{\text{plan}}(\pi_{\text{student}}^{\theta}(o_i^{\text{pix}}), \tau_i) \right],$$

where $\mathcal{L}_{\text{plan}}$ scores the student’s predicted trajectory $\hat{\tau}_i = \pi_{\text{student}}^{\theta}(o_i^{\text{pix}})$ against the target τ_i .¹ Because the teacher is conditioned on a per-agent reward-preference vector $\mathbf{c}_i$, the target τ_i is persona-specific. We therefore condition the student on the same $\mathbf{c}_i$ and match it to the teacher during distillation, so the target is consistent with the student’s inputs; sampling $\mathbf{c}_i$ additionally exposes the student to diverse behaviors during training. We suppress $\mathbf{c}_i$ in the notation above for brevity; full conditioning details are provided in the Appendix.

3.3 Sim-to-real Perception Adaptation

Self-play DAgger yields a student π_{student} that drives robustly in closed loop on Gigapixel’s abstract renderings o_i^{pix} , but deploying on real sensor data requires closing the gap between these renderings and real camera images. We frame this as an image-to-image *perceptual adaptation* task [92, 93]: rather than retraining the full policy, we adapt only the perception stack so that real images map into the same latent representation the planning head already acts on. We curate a dataset of paired observations $\mathcal{D}_{\text{paired}} = \{(o^{\text{real}}, o^{\text{pix}})\}$, where o^{pix} is the Gigapixel rendering of the abstract state reconstructed from the real log corresponding to o^{real} . From the trained student weights $\theta = (\theta_{\text{per}}, \theta_{\text{plan}})$ we instantiate two copies sharing the frozen planning head $f_{\theta_{\text{plan}}}$: a frozen teacher π^{sim} on o^{pix} and a trainable student π^{real} on o^{real} , whose perception backbone $f_{\theta_{\text{per}}^{\text{real}}}$ is the only module updated. π^{sim} supervises π^{real} at both the output and feature levels via the planning loss $\mathcal{L}_{\text{plan}}$ used in self-play DAgger and a perceptual loss $\mathcal{L}_{\text{perc}}(E_{\text{per}}^{\text{real}}, E_{\text{per}}^{\text{sim}}) = \|E_{\text{per}}^{\text{real}} - E_{\text{per}}^{\text{sim}}\|_2^2$ that aligns the adapted features $E_{\text{per}}^{\text{real}} = f_{\theta_{\text{per}}^{\text{real}}}(o^{\text{real}})$ with the frozen reference $E_{\text{per}}^{\text{sim}} = f_{\theta_{\text{per}}^{\text{sim}}}(o^{\text{pix}})$. The full objective is

$$\min_{\theta_{\text{per}}^{\text{real}}} \mathbb{E}_{(o^{\text{real}}, o^{\text{pix}}) \sim \mathcal{D}_{\text{paired}}} \left[\mathcal{L}_{\text{plan}}(\pi^{\text{real}}(o^{\text{real}}), \pi^{\text{sim}}(o^{\text{pix}})) + \lambda \mathcal{L}_{\text{perc}}(E_{\text{per}}^{\text{real}}, E_{\text{per}}^{\text{sim}}) \right],$$

¹We leave $\mathcal{L}_{\text{plan}}$ deliberately general: it may be any planning loss compatible with the planning head $f_{\theta_{\text{plan}}}$.

Method	Human Traj?	SimScale Data?	Stage	NC $\uparrow$	DAC $\uparrow$	DDC $\uparrow$	TLC $\uparrow$	EP $\uparrow$	TTC $\uparrow$	LK $\uparrow$	HC $\uparrow$	EC $\uparrow$	Per-Stage Score $\uparrow$	EPDMS $\uparrow$
LTF [3]	✓	✗	S 1 S 2	96.2 77.8	79.6 70.2	99.1 84.3	99.6 98.1	84.1 85.1	95.1 75.7	94.2 45.4	97.6 95.7	79.1 76.0	- -	25.1
RAP [71]	✓	✗	S 1 S 2	97.1 83.2	94.4 83.9	98.8 87.4	99.8 98.0	83.9 86.9	96.9 80.4	94.7 52.3	96.4 95.2	66.2 52.4	- -	39.6
ZTRS [94]	✗	✗	S 1 S 2	98.9 91.1	97.6 90.4	100.0 95.8	100.0 99.0	66.7 63.6	98.9 89.8	96.2 60.4	96.7 97.6	44.0 66.1	- -	48.1
GuideFlow [95]	✓	✗	S 1 S 2	99.6 91.4	98.0 89.5	99.4 95.2	99.3 98.9	79.7 77.5	99.3 89.6	94.9 52.6	97.1 93.6	58.2 51.0	- -	51.5
SimScale [96]	✓	✓	S 1 S 2	99.6 94.5	99.1 94.2	99.9 95.8	100.0 99.2	69.6 75.8	99.6 92.8	95.8 60.1	95.6 96.1	28.4 43.2	- -	53.2
<i>Regression-based (Single-Mode) DrivoR Planner</i>														
DrivoR-Reg [21]	✓	✗	S 1 S 2	96.1 79.0	76.4 68.5	97.6 82.2	99.3 98.7	83.3 83.4	95.6 76.9	93.1 44.0	97.8 95.9	80.4 77.6	65.2 38.4	25.5
Gigapixel-DrivoR-Reg	✗	✗	S 1 S 2	90.2 80.4	78.9 77.4	95.3 86.7	99.3 96.8	86.4 91.4	88.4 77.6	95.8 57.7	97.8 97.4	64.4 52.1	60.9 45.5	29.5
<i>Scoring-based (Multimodal) DrivoR Planner</i>														
DrivoR [21]	✓	✗	S 1 S 2	98.8 90.2	95.1 88.4	98.9 91.9	100 98.6	72.6 70.0	98.7 88.0	94.0 50.1	97.6 98.5	73.3 76.2	80.9 59.4	48.3
DrivoR [21] (w/ SimScale)	✓	✓	S 1 S 2	99.1 92.3	98.2 91.6	99.3 97.3	99.8 99.1	75.4 75.7	98.7 90.6	94.9 56.1	97.6 98.4	70.2 44.7	84.2 64.6	54.7
Gigapixel-DrivoR	✗	✗	S 1 S 2	99.4 93.7	95.8 92.9	99.4 96.0	99.8 98.9	68.1 62.6	99.6 90.7	91.8 60.2	97.6 98.2	49.8 58.8	77.8 63.5	50.1

Table 2: NAVSIM-v2 navhard **Results**. Gigapixel models perform competitively across Stage 1 metrics without human trajectory supervision, and further improves Stage 2 performance, which we emphasize as the closest proxy for closed-loop execution robustness; see text for discussion.

with λ weighting the perceptual loss. This transfers π_{student} 's closed-loop behavior to real images using only paired observations, without human trajectory supervision.

4 Experiments

Implementation Details. We train policies in Gigapixel using 335k 20s scenarios uniformly sampled from the nuPlan train split [30], extracting agent states, static objects, lane polylines, and traffic-light states. During self-play, vehicles are policy-controlled, while pedestrians, cyclists, and traffic lights are log-replayed. The privileged vectorized teacher π_{teacher} follows our re-implementation of Gigaflow [16]: a compact 2.7M-parameter permutation-invariant policy trained for 25B agent steps in the extracted nuPlan scenarios. We train two pixel-based student architectures: a scoring-based DrivoR model [21], which predicts 64 trajectory proposals and selects the one with highest predicted PDMS, and a regression-only variant, DrivoR-Reg, which outputs a single trajectory without a scoring head. Both students are trained with self-play DAGger in Gigapixel for 150M steps, then adapted from simulated renderings to real NAVSIM camera images using paired simulated-real observations from navtrain. Further training details and hyperparameters are provided in the Appendix.

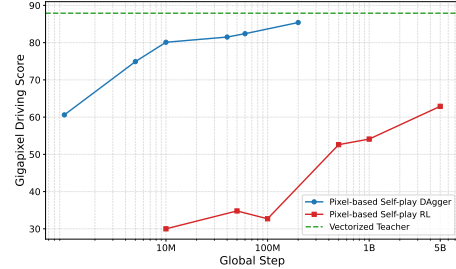


Figure 3: **Self-play DAGger vs. RL**. We compare two pixel-based training methods, self-play DAGger and self-play RL, with a CNN-based model in Gigapixel. We plot Gigapixel Driving Score (Completion Rate – Offroad Rate – Collision Rate) on a held-out set as a function of agent steps in Gigapixel. The dashed line marks the vectorized teacher performance at 25B steps.

Benchmarks and Evaluated Methods. We evaluate in Gigapixel and on two real-world driving benchmarks: HUGSIM [27] and NAVSIM-v2 [28]. In Gigapixel, we run closed-loop evaluation on 1,000 held-out nuPlan scenarios with log-replayed surrounding actors, reporting the Gigapixel Driving Score $\max(0, \text{Completion Rate} - \text{Collision Rate} - \text{Off-road Rate})$. HUGSIM evaluates pixel-based policies in closed-loop on reconstructed real-world scenes using HD-Score, reported across Easy, Medium, Hard, and Extreme difficulty tiers. NAVSIM-v2 evaluates pseudo-closed-loop robustness using EPDMS on the navhard split; we emphasize Stage 2 EPDMS as the closest proxy

Gigapixel Training Strategy	Perception L2 Loss	Frozen Planning Head?	HD-Score				
			E	M	H	X	Avg.
Self-play DAgger	✗	✗	43.5	10.5	7.3	16.4	15.8
Self-play DAgger	✗	✓	52.6	15.1	7.2	15.2	18.5
DAgger	✓	✓	61.8	32.0	15.1	24.3	<u>30.1</u>
BC	✓	✓	45.2	22.7	10.9	4.9	18.6
Self-play DAgger	✓	✓	65.8	38.8	13.1	24.6	33.2

Table 3: **Ablation study.** We ablate the Gigapixel training strategy, and perception adaptation design on HUGSIM. All variants use DrivoR-Reg. Best method is **bolded**; second-best underlined.

for recovery under closed-loop execution, as it measures performance in perturbed ego poses. We evaluate two self-play trained policies, *Gigapixel-DrivoR* and *Gigapixel-DrivoR-Reg*, obtained by training DrivoR and DrivoR-Reg in Gigapixel with self-play DAgger followed by sim-to-real perception adaptation. We compare against behavior-cloned DrivoR variants, DrivoR with SimScale recovery data, published leaderboard methods, and ablations that replace self-play DAgger with behavior cloning or single-agent DAgger. Full benchmark protocols and metrics, baseline training details, and metric definitions are provided in the Appendix.

Results. Figure 1 compares Gigapixel’s rendering throughput against alternative pixel renderers. The Gigapixel rasterizer is $\sim 1000\times$ faster than the Gaussian-splatting-based HUGSIM [27] and $\sim 4000\times$ faster than the rasterization-based RAP [71] at the 512×512 resolution. Although RAP adopts a similar simplified scene representation, it runs entirely on CPU and forgoes the batched entity-component-system (ECS) design that underpins Madrona’s high throughput [72]. Figure 3 compares self-play DAgger training against direct self-play RL training from pixels for an action-based CNN policy in Gigapixel. We use the CNN policy as it is a lighter weight architecture than DrivoR, thus enabling higher throughput training for RL experimentation (see Figure 1). Self-play DAgger is dramatically more sample-efficient: it surpasses a Gigapixel Driving Score of 60 in roughly $3000\times$ fewer steps than self-play RL, motivating our DAgger-based approach.

Table 1 reports closed-loop driving performance on the HUGSIM benchmark. Gigapixel-DrivoR achieves state-of-the-art performance, outperforming all baselines on both average RC (50.1) and HD-Score (38.5)—without any human trajectory supervision. The scoring-based Gigapixel-DrivoR improves over its behavior-cloning counterpart DrivoR by 2.8 HD-Score points (38.5 vs. 35.7), and the regression-based Gigapixel-DrivoR-Reg improves over DrivoR-Reg by 12.5 points (33.2 vs. 20.7)—a 60% relative gain that demonstrates the impact of closed-loop self-play DAgger training. The one regime where DrivoR exceeds Gigapixel-DrivoR is the Extreme tier (32.5 vs. 21.6 HD-Score), where surrounding actors are most adversarial. Manual inspection (examples in the supplementary) reveals a *high-velocity bias* in DrivoR: it tends to drive fast, which incidentally allows it to outpace adversarial actors attempting to induce collisions. Gigapixel-DrivoR drives more cautiously and instead yields to these actors, often getting stuck. To quantify this, we measured the average collision velocity across the full evaluation set: DrivoR collides at 5.27 m/s on average, compared to 1.95 m/s for Gigapixel-DrivoR—a $2.7\times$ reduction, indicating that DrivoR’s Extreme-tier advantage stems from a less safe driving style rather than better closed-loop reasoning.

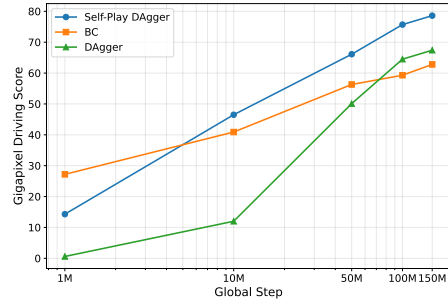


Figure 4: **Scaling Experiments.** We compare different end-to-end training strategies at scale in Gigapixel. We plot Gigapixel Driving Score (Completion Rate – Offroad Rate – Collision Rate) on a held-out set as a function of agent steps in Gigapixel. In all configurations, we train the single-mode DrivoR-Reg architecture.



Figure 5: **Qualitative comparison of closed-loop driving behavior near a decelerating lead vehicle.** Frames advance left to right ($t_1 \rightarrow t_3$); the yellow waypoints denote the model’s planned future trajectory. **Top (green): Self-Play Trained DrivoR.** As the lead vehicle comes to a stop, the policy reduces speed and plans a smooth lateral trajectory that nudges out of the lane and passes the stationary vehicle safely. **Bottom (red): BC Trained DrivoR.** The behavior-cloned policy keeps a centered straight-ahead plan, closes distance on the lead vehicle, and ultimately rear-ends it.

Table 2 reports pseudo-closed-loop performance on NAVSIM-v2 navhard. Without human trajectory supervision or SimScale data, Gigapixel-DrivoR and Gigapixel-DrivoR-Reg outperform their behavior-cloning counterparts in EPDMS (50.1 vs. 48.3 and 29.5 vs. 25.5, respectively). These gains are concentrated in Stage 2, which evaluates planning from perturbed off-distribution ego poses and is therefore the closest proxy for closed-loop robustness: Gigapixel-DrivoR-Reg improves over DrivoR-Reg by 7.1 points (45.5 vs. 38.4), while Gigapixel-DrivoR improves over DrivoR by 4.1 points (63.5 vs. 59.4). DrivoR (w/ SimScale) achieves the best overall EPDMS (54.7), but SimScale’s data curation explicitly mines high-EPDMS trajectories, whereas our Gigaflo teacher is not trained with a NAVSIM-v2-specific objective. Even so, Gigapixel-DrivoR nearly matches its Stage 2 score (63.5 vs. 64.6).

Figure 4 reports closed-loop performance of the DrivoR-Reg student across training strategies as experience scales. Self-play DAGger improves consistently with scale and outperforms both single-agent DAGger and behavior cloning (BC) beyond 10M steps. BC plateaus around 100M steps, since the student is never exposed to states from its own rollouts. Self-play DAGger also outperforms single-agent DAGger throughout, reflecting the two factors identified in Section 3.2: every agent in a rollout contributes supervised data, and the resulting interactions span more diverse multi-agent behaviors. Table 3 confirms these gains transfer after sim-to-real perception adaptation on the HUGSIM benchmark: self-play DAGger achieves a 3.1-point HD-Score improvement over single-agent DAGger and 14.6 points over BC, mirroring the in-simulation ranking. The table also ablates our adaptation design: removing the perceptual loss drops HD-Score from 33.2 to 18.5, and additionally unfreezing the planning head drops it to 15.8—confirming that both choices are essential for preserving the closed-loop behavior learned during self-play.

Figure 5 contrasts the two policies as the ego vehicle approaches a lead vehicle slowing to a stop. The self-play trained DrivoR (top row) anticipates the deceleration, reduces speed, and once the lead vehicle stops plans a safe collision-free lateral maneuver around it. The BC trained DrivoR (bottom row), by contrast, maintains a high-velocity straight-ahead trajectory ($t_1 \rightarrow t_3$) and ultimately rear-ends the stopped vehicle. The failure exposes a limitation of behavior cloning: the rare, safety-critical states that precede a stop are underrepresented at training time, so the policy never learns the deceleration-and-avoid response that self-play discovers through explicit closed-loop interaction.

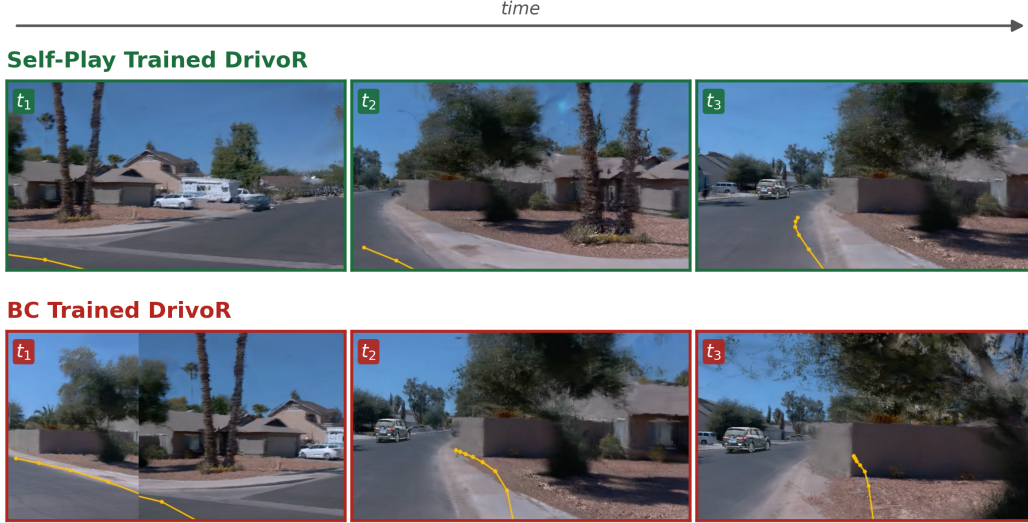


Figure 6: **Behavior from a recovery state near the road edge.** Frames advance in time ($t_1 \rightarrow t_3$); yellow waypoints denote the planned trajectory. **Top (green): Self-Play Trained DrivoR** plans a slow, careful turn that corrects toward the lane and keeps the vehicle on the drivable surface. **Bottom (red): BC Trained DrivoR** fails to correct adequately near the road edge and drifts off-road.

Figure 6 compares the two policies initialized in a recovery state, where the vehicle starts near the road boundary and a slow, careful turn is required to avoid going off-road. The self-play trained DrivoR (top row) reduces speed and plans a corrective trajectory, recovering safely to a nominal lane position. The BC trained DrivoR (bottom row) does not steer enough near the edge, and its trajectory drifts off the drivable surface, resulting in an off-road violation. Such corrective states are frequently visited during self-play training, but are rarely present in offline expert logs.

5 Conclusion, Limitations, and Future Work

We present the first method for effectively training end-to-end driving policies via self-play. Our approach combines Gigapixel for high-throughput pixel-based simulation, self-play DAgger for distilling a privileged RL teacher, and lightweight sim-to-real adaptation, achieving state-of-the-art HUGSIM performance and strong NAVSIM-v2 performance. These contributions establish self-play as a viable alternative to large-scale behavior cloning for training end-to-end driving models.

Limitations. Gigapixel is limited by its abstract scene representation: agent boxes, lane polylines, and traffic-light states cannot capture cues such as debris, unusual obstacles, weather, or lighting. It also introduces a teacher-student asymmetry [97], where the teacher observes privileged vectorized observations o^{vec} while the student must infer actions from pixels o^{pix} , which can be unrecoverable under occlusion or limited visibility. Finally, sim-to-real adaptation requires paired data $\mathcal{D}_{\text{paired}}$, which is practical on NAVSIM but less direct with only raw sensor logs.

Future Work. Several directions follow naturally from this work. First, while we deliberately forgo human trajectory supervision to demonstrate the strength of self-play alone, the most capable real-world policies will likely combine both signals. Human data could be incorporated through KL-regularization during self-play training [18] or through post-training toward human-like behavior. Second, Gigapixel currently initializes scenarios from nuPlan logs, which constrains the distribution of initial scene configurations. Generative approaches [98, 99] could synthesize initial conditions that induce rare or informative interactions, further enriching the self-play curriculum. Finally, training pixel-based RL policies directly in Gigapixel with native trajectory outputs remains an open challenge, but would eliminate the teacher-student asymmetry inherent in distillation approaches.

Acknowledgments

We thank Samsung, the IVADO and the Canada First Research Excellence Fund (CFREF) / Apogee Funds, the Canada CIFAR AI Chairs Program, Fonds de Recherche Nature et Technologies (FRQNT), and the NSERC Discovery Grants and Doctoral Scholarship programs for their financial support. We thank Mila - Quebec AI Institute for compute resources. Daphne Cornelisse is partially supported by the Cooperative AI Foundation and a Chishiki-AI SCIPE Fellowship.

References

- [1] P. S. Chib and P. Singh. Recent advancements in end-to-end autonomous driving using deep learning: A survey. *IEEE Transactions on Intelligent Vehicles*, 9(1):103–118, 2023.
- [2] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li. End-to-end autonomous driving: Challenges and frontiers. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):10164–10183, 2024.
- [3] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger. Transfuser: Imitation with transformer-based sensor fusion for autonomous driving. *IEEE transactions on pattern analysis and machine intelligence*, 45(11):12878–12895, 2022.
- [4] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, et al. Planning-oriented autonomous driving. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 17853–17862, 2023.
- [5] B. Jiang, S. Chen, Q. Xu, B. Liao, J. Chen, H. Zhou, Q. Zhang, W. Liu, C. Huang, and X. Wang. Vad: Vectorized scene representation for efficient autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8340–8350, 2023.
- [6] J.-J. Hwang, R. Xu, H. Lin, W.-C. Hung, J. Ji, K. Choi, D. Huang, T. He, P. Covington, B. Sapp, et al. Emma: End-to-end multimodal model for autonomous driving. *arXiv preprint arXiv:2410.23262*, 2024.
- [7] L. Rowe, R. de Schaetzen, R. Girgis, C. Pal, and L. Paull. Poutine: Vision-language-trajectory pre-training and reinforcement learning post-training enable robust end-to-end autonomous driving. *arXiv preprint arXiv:2506.11234*, 2025.
- [8] Y. Zheng, P. Yang, Z. Xia, Q. Zhang, Y. Zheng, S. Gu, B. Jin, T. Zhang, B. Lu, C. Han, et al. Data scaling laws for imitation learning-based end-to-end autonomous driving. *arXiv preprint arXiv:2412.02689*, 2024.
- [9] A. Naumann, X. Gu, T. Dimlioglu, M. Bojarski, A. Degirmenci, A. Popov, D. Bisla, M. Pavone, U. Muller, and B. Ivanovic. Data scaling laws for end-to-end autonomous driving. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 2571–2582, 2025.
- [10] M. Baniodeh, K. Goel, S. Ettinger, C. Fuertes, A. Seff, T. Shen, C. Gulino, C. Yang, G. Jerfel, D. Choe, et al. Scaling laws of motion forecasting and planning—technical report. *arXiv preprint arXiv:2506.08228*, 2025.
- [11] L. Le Mero, D. Yi, M. Dianati, and A. Mouzakitis. A survey on imitation learning techniques for end-to-end autonomous vehicles. *IEEE Transactions on Intelligent Transportation Systems*, 23(9):14128–14147, 2022.
- [12] W. Ljungbergh, A. Tonderski, J. Johnander, H. Caesar, K. Åström, M. Felsberg, and C. Petersson. Neuroncap: Photorealistic closed-loop safety testing for autonomous driving. *European Conference on Computer Vision (ECCV)*, 2024.

- [13] P. Karkus, M. Igl, Y. Chen, K. Chitta, J. Packer, B. Douillard, R. Tian, A. Naumann, G. Garcia-Cobo, S. Tan, et al. Beyond behavior cloning in autonomous driving: a survey of closed-loop training techniques. *Authorea Preprints*, 2025.
- [14] S. Ross, G. Gordon, and D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.
- [15] F. Codevilla, E. Santana, A. M. López, and A. Gaidon. Exploring the limitations of behavior cloning for autonomous driving. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9329–9338, 2019.
- [16] M. Cusumano-Towner, D. Hafner, A. Hertzberg, B. Huval, A. Petrenko, E. Vinitsky, E. Wijmans, T. W. Killian, S. Bowers, O. Sener, et al. Robust autonomy emerges from self-play. In *International Conference on Machine Learning*, pages 11710–11737. PMLR, 2025.
- [17] D. Cornelisse, S. Cheng, P. Mandavilli, J. Hunt, K. Joseph, W. Doulazmi, V. Charraut, A. Gupta, J. Suarez, and E. Vinitsky. PufferDrive: A fast and friendly driving simulator for training and evaluating RL agents, 2025. URL <https://github.com/Emerge-Lab/PufferDrive>.
- [18] D. Cornelisse and E. Vinitsky. Human-compatible driving agents through data-regularized self-play reinforcement learning. In *Reinforcement Learning Conference*, 2024.
- [19] D. Cornelisse, A. Pandya, K. Joseph, J. Suárez, and E. Vinitsky. Building reliable sim driving agents by scaling self-play. *arXiv preprint arXiv:2502.14706*, 2025.
- [20] C. Zhang, S. Biswas, K. Wong, K. Fallah, L. Zhang, D. Chen, S. Casas, and R. Urtasun. Learning to drive via asymmetric self-play. In *European Conference on Computer Vision*, pages 149–168. Springer, 2024.
- [21] E. Kirby, A. Boulch, Y. Xu, Y. Yin, G. Puy, É. Zablocki, A. Bursuc, S. Gidaris, R. Marlet, F. Bartoccioni, et al. Driving on registers. *arXiv preprint arXiv:2601.05083*, 2026.
- [22] L. G. Rosenzweig, B. Shacklett, W. Xia, and K. Fatahalian. High-throughput batch rendering for embodied ai. In *SIGGRAPH Asia 2024 Conference Papers*, pages 1–9, 2024.
- [23] R. Agarwal, N. Vieillard, Y. Zhou, P. Stanczyk, S. R. Garea, M. Geist, and O. Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *The twelfth international conference on learning representations*, 2024.
- [24] Y. Wang, W. Luo, J. Bai, Y. Cao, T. Che, K. Chen, Y. Chen, J. Diamond, Y. Ding, W. Ding, et al. Alpamayo-r1: Bridging reasoning and action prediction for generalizable autonomous driving in the long tail. *arXiv preprint arXiv:2511.00088*, 2025.
- [25] B. Liao, S. Chen, H. Yin, B. Jiang, C. Wang, S. Yan, X. Zhang, X. Li, Y. Zhang, Q. Zhang, et al. Diffusiondrive: Truncated diffusion model for end-to-end autonomous driving. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 12037–12047, 2025.
- [26] B. Jaeger, D. Dauner, J. Beißwenger, S. Gerstenecker, K. Chitta, and A. Geiger. Carl: Learning scalable planning policies with simple rewards. *arXiv preprint arXiv:2504.17838*, 2025.
- [27] H. Zhou, L. Lin, J. Wang, Y. Lu, D. Bai, B. Liu, Y. Wang, A. Geiger, and Y. Liao. Hugsim: A real-time, photo-realistic and closed-loop simulator for autonomous driving. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.

- [28] W. Cao, M. Hallgarten, T. Li, D. Dauner, X. Gu, C. Wang, Y. Miron, M. Aiello, H. Li, I. Gilitschenski, et al. Pseudo-simulation for autonomous driving. In *Conference on Robot Learning*, pages 4709–4722. PMLR, 2025.
- [29] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom. nuscenes: A multimodal dataset for autonomous driving. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11621–11631, 2020.
- [30] H. Caesar, J. Kabzan, K. S. Tan, W. K. Fong, E. Wolff, A. Lang, L. Fletcher, O. Beijbom, and S. Omari. nuplan: A closed-loop ml-based planning benchmark for autonomous vehicles. *arXiv preprint arXiv:2106.11810*, 2021.
- [31] P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, et al. Scalability in perception for autonomous driving: Waymo open dataset. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2446–2454, 2020.
- [32] R. Xu, H. Lin, W. Jeon, H. Feng, Y. Zou, L. Sun, J. Gorman, E. Tolstaya, S. Tang, B. White, et al. Wod-e2e: Waymo open dataset for end-to-end driving in challenging long-tail scenarios. *arXiv preprint arXiv:2510.26125*, 2025.
- [33] H. Arai, K. Miwa, K. Sasaki, K. Watanabe, Y. Yamaguchi, S. Aoki, and I. Yamamoto. Covla: Comprehensive vision-language-action dataset for autonomous driving. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1933–1943. IEEE, 2025.
- [34] F. Ghilotti, E. Palladin, S. Brucker, A. Sigal, M. Bijelic, and F. Heide. Truckdrive: Long-range autonomous highway driving dataset. *arXiv preprint arXiv:2603.02413*, 2026.
- [35] X. Weng, B. Ivanovic, Y. Wang, Y. Wang, and M. Pavone. Para-drive: Parallelized architecture for real-time autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15449–15458, 2024.
- [36] Z. Li, K. Li, S. Wang, S. Lan, Z. Yu, Y. Ji, Z. Li, Z. Zhu, J. Kautz, Z. Wu, et al. Hydramdp: End-to-end multimodal planning with multi-target hydra-distillation. *arXiv preprint arXiv:2406.06978*, 2024.
- [37] Z. Xing, X. Zhang, Y. Hu, B. Jiang, T. He, Q. Zhang, X. Long, and W. Yin. Goalflow: Goal-driven flow matching for multimodal trajectories generation in end-to-end autonomous driving. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 1602–1611, 2025.
- [38] K. Renz, L. Chen, E. Arani, and O. Sinavski. Simlingo: Vision-only closed-loop autonomous driving with language-action alignment. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 11993–12003, 2025.
- [39] Z. Zhou, T. Cai, S. Zhao, Y. Zhang, Z. Huang, B. Zhou, and J. Ma. Autovla: A vision-language-action model for end-to-end autonomous driving with adaptive reasoning and reinforcement fine-tuning. *Advances in Neural Information Processing Systems*, 38:27920–27956, 2026.
- [40] P. De Haan, D. Jayaraman, and S. Levine. Causal confusion in imitation learning. *Advances in neural information processing systems*, 32, 2019.
- [41] D. Dauner, M. Hallgarten, A. Geiger, and K. Chitta. Parting with misconceptions about learning-based vehicle motion planning. In *Conference on Robot Learning*, pages 1268–1281. PMLR, 2023.

- [42] B. R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A. A. Al Sallab, S. Yogamani, and P. Pérez. Deep reinforcement learning for autonomous driving: A survey. *IEEE transactions on intelligent transportation systems*, 23(6):4909–4926, 2021.
- [43] J. Chen, B. Yuan, and M. Tomizuka. Model-free deep reinforcement learning for urban autonomous driving. In *2019 IEEE intelligent transportation systems conference (ITSC)*, pages 2765–2771. IEEE, 2019.
- [44] M. Toromanoff, E. Wirbel, and F. Moutarde. End-to-end model-free reinforcement learning for urban driving using implicit affordances. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 7153–7162, 2020.
- [45] R. Chekroun, M. Toromanoff, S. Hornauer, and F. Moutarde. Gri: General reinforced imitation and its application to vision-based autonomous driving. *Robotics*, 12(5):127, 2023.
- [46] C. Zhang, R. Guo, W. Zeng, Y. Xiong, B. Dai, R. Hu, M. Ren, and R. Urtasun. Rethinking closed-loop training for autonomous driving. In *European Conference on Computer Vision*, pages 264–282. Springer, 2022.
- [47] H. Gao, S. Chen, B. Jiang, B. Liao, Y. Shi, X. Guo, Y. Pu, X. Li, W. Liu, Q. Zhang, et al. Rad: Training an end-to-end driving policy via large-scale 3dgs-based reinforcement learning. *Advances in Neural Information Processing Systems*, 38:32551–32576, 2026.
- [48] C. Ni, G. Zhao, X. Wang, Z. Zhu, W. Qin, X. Chen, G. Jia, G. Huang, and W. Mei. Recondreamer-rl: Enhancing reinforcement learning via diffusion-based scene reconstruction. *arXiv preprint arXiv:2508.08170*, 2025.
- [49] Z. Zhang, P. Karkus, M. Igl, W. Ding, Y. Chen, B. Ivanovic, and M. Pavone. Closed-loop supervised fine-tuning of tokenized traffic models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 5422–5432, 2025.
- [50] J. Zhang and K. Cho. Query-efficient imitation learning for end-to-end autonomous driving. *arXiv preprint arXiv:1605.06450*, 2016.
- [51] A. Prakash, A. Behl, E. Ohn-Bar, K. Chitta, and A. Geiger. Exploring data aggregation in policy learning for vision-based urban autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11763–11773, 2020.
- [52] G. Garcia-Cobo, M. Igl, P. Karkus, Z. Zhang, M. Watson, Y. Chen, B. Ivanovic, and M. Pavone. Road: Rollouts as demonstrations for closed-loop supervised fine-tuning of autonomous driving policies. *arXiv preprint arXiv:2512.01993*, 2025.
- [53] S. Z. Zhao, L. Wang, H. Ruan, Y. Bao, Y. Chen, Z. Leng, A. Ravichandran, H. He, Z. Zhou, X. Han, et al. Bridgesim: Unveiling the ol-cl gap in end-to-end autonomous driving. *arXiv preprint arXiv:2604.10856*, 2026.
- [54] A. Popov, A. Degirmenci, D. Wehr, S. Hegde, R. Oldja, A. Kamenev, B. Douillard, D. Nistér, U. Muller, R. Bhargava, et al. Mitigating covariate shift in imitation learning for autonomous vehicles using latent space generative world models. *arXiv preprint arXiv:2409.16663*, 2024.
- [55] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun. CARLA: An open urban driving simulator. In *Conference on Robot Learning*, pages 1–16. PMLR, 2017.
- [56] D. Coelho, M. Oliveira, and V. Santos. Rlad: Reinforcement learning from pixels for autonomous driving in urban environments. *IEEE Transactions on Automation Science and Engineering*, 21(4):7427–7435, 2023.
- [57] B. Osiński, A. Jakubowski, P. Zięcina, P. Miłoś, C. Galias, S. Homocanu, and H. Michalewski. Simulation-based reinforcement learning for real-world autonomous driving. In *2020 IEEE international conference on robotics and automation (ICRA)*, pages 6411–6418. IEEE, 2020.

- [58] C. Gulino, J. Fu, W. Luo, G. Tucker, E. Bronstein, Y. Lu, J. Harb, X. Pan, Y. Wang, X. Chen, et al. Waymax: An accelerated, data-driven simulator for large-scale autonomous driving research. *Advances in Neural Information Processing Systems*, 36, 2023.
- [59] E. Vinitsky, N. Lichtlé, X. Yang, B. Amos, and J. Foerster. Nocturne: a scalable driving benchmark for bringing multi-agent learning one step closer to the real world. *Advances in Neural Information Processing Systems*, 35:3962–3974, 2022.
- [60] S. Kazemkhani, A. Pandya, D. Cornelisse, B. Shacklett, and E. Vinitsky. GPUDrive: Data-driven, multi-agent driving simulation at 1 million FPS. In *The Thirteenth International Conference on Learning Representations, ICLR 2025*, 2025.
- [61] B. Wymann, E. Espié, C. Guionneau, C. Dimitrakakis, R. Coulom, and A. Sumner. Torcs, the open racing car simulator. *Software available at <http://torcs.sourceforge.net>*, 4(6):2, 2000.
- [62] Q. Li, Z. Peng, L. Feng, Q. Zhang, Z. Xue, and B. Zhou. Metadrive: Composing diverse driving scenarios for generalizable reinforcement learning. *IEEE transactions on pattern analysis and machine intelligence*, 45(3):3461–3475, 2022.
- [63] Z. Yang, Y. Chen, J. Wang, S. Manivasagam, W.-C. Ma, A. J. Yang, and R. Urtasun. Unisim: A neural closed-loop sensor simulator. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1389–1399, 2023.
- [64] A. Tonderski, C. Lindström, G. Hess, W. Ljungbergh, L. Svensson, and C. Petersson. Neurad: Neural rendering for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14895–14904, 2024.
- [65] NVIDIA, Y. Cao, R. de Lutio, S. Fidler, G. G. Cobo, Z. Gojcic, M. Igl, B. Ivanovic, P. Karkus, J. M. Esturo, M. Pavone, A. Smith, E. Tanimura, M. Tyszkiewicz, M. Watson, Q. Wu, and L. Zhang. Alpasim: A modular, lightweight, and data-driven research simulator for autonomous driving, October 2025. URL <https://github.com/NVlabs/alphasim>.
- [66] A. Hu, L. Russell, H. Yeo, Z. Murez, G. Fedoseev, A. Kendall, J. Shotton, and G. Corrado. Gaia-1: A generative world model for autonomous driving. *arXiv preprint arXiv:2309.17080*, 2023.
- [67] L. Russell, A. Hu, L. Bertoni, G. Fedoseev, J. Shotton, E. Arani, and G. Corrado. Gaia-2: A controllable multi-view generative world model for autonomous driving. *arXiv preprint arXiv:2503.20523*, 2025.
- [68] J. Yang, K. Chitta, S. Gao, L. Chen, Y. Shao, X. Jia, H. Li, A. Geiger, X. Yue, and L. Chen. ReSim: Reliable world simulation for autonomous driving. *arXiv preprint arXiv:2506.09981*, 2025.
- [69] X. Yang, L. Wen, T. Wei, Y. Ma, J. Mei, X. Li, W. Lei, D. Fu, P. Cai, M. Dou, et al. Drivearena: A closed-loop generative simulation platform for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 26933–26943, 2025.
- [70] T. Yan, D. Wu, W. Han, J. Jiang, X. Zhou, K. Zhan, C.-z. Xu, and J. Shen. Drivingsphere: Building a high-fidelity 4d world for closed-loop simulation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 27531–27541, 2025.
- [71] L. Feng, Y. Gao, E. Zablocki, Q. Li, W. Li, S. Liu, M. Cord, and A. Alahi. RAP: 3d rasterization augmented end-to-end planning. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=a9b0geqbdB>.
- [72] B. Shacklett, L. G. Rosenzweig, Z. Xie, B. Sarkar, A. Szot, E. Wijmans, V. Koltun, D. Batra, and K. Fatahalian. An extensible, data-oriented architecture for high-performance, many-world simulation. *ACM Transactions on Graphics (TOG)*, 42(4):1–13, 2023.

- [73] H. X. Liu and S. Feng. Curse of rarity for autonomous vehicles. *nature communications*, 15 (1):4808, 2024.
- [74] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.
- [75] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis. Mastering the game of Go without human knowledge. *Nature*, 550(7676):354–359, 2017.
- [76] M. Jaderberg, W. M. Czarnecki, I. Dunning, L. Marris, G. Lever, A. García Castañeda, C. Beattie, N. C. Rabinowitz, A. S. Morcos, A. Ruderman, N. Sonnerat, T. Green, L. Deason, J. Z. Leibo, D. Silver, D. Hassabis, K. Kavukcuoglu, and T. Graepel. Human-level performance in first-person multiplayer games with population-based deep reinforcement learning. *arXiv preprint arXiv:1807.01281*, 2018.
- [77] A. Bakhtin, D. J. Wu, A. Lerer, J. Gray, A. P. Jacob, G. Farina, A. H. Miller, and N. Brown. Mastering the game of no-press diplomacy via human-regularized reinforcement learning and planning. In *The Eleventh International Conference on Learning Representations, ICLR 2023*, 2023.
- [78] W.-J. Chang, A. Ranges, K. Joseph, M. Strong, M. Tomizuka, Y. Hu, and W. Zhan. Spacer: Self-play anchoring with centralized reference models. *arXiv preprint arXiv:2510.18060*, 2025.
- [79] Z. Wang, S. Rahmani, D. Cornelisse, B. Sarkar, A. D. Goldie, J. N. Foerster, and S. Whiteson. Learning to drive in new cities without human demonstrations. *arXiv preprint arXiv:2602.15891*, 2026.
- [80] A. Distelzweig, F. Janjoš, A. Look, A. Rothenhäusler, D. Jost, O. Scheel, R. Rajan, D. Cornelisse, E. Vinitsky, and J. Boedecker. Beyond self-play and scale: A behavior benchmark for generalization in autonomous driving. *arXiv preprint arXiv:2605.10034*, 2026.
- [81] J. Qiu, A. Saviolo, C. Wang, M. Wang, and X. Huang. Heterogeneous self-play for realistic highway traffic simulation. *arXiv preprint arXiv:2604.16406*, 2026.
- [82] R. S. Sutton, A. G. Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- [83] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12, 1999.
- [84] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.
- [85] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897. PMLR, 2015.
- [86] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [87] J. D. Foley. *Computer graphics: principles and practice*, volume 12110. Addison-Wesley Professional, 1996.

- [88] A. S. Glassner. *An introduction to ray tracing*. Morgan Kaufmann, 1989.
- [89] D. M. Saxena, S. Bae, A. Nakhaei, K. Fujimura, and M. Likhachev. Driving in dense traffic with model-free reinforcement learning. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5385–5392. IEEE, 2020.
- [90] M. Harmel, A. Paras, A. Pasternak, N. Roy, and G. Linscott. Scaling is all you need: Autonomous driving with jax-accelerated reinforcement learning. *arXiv preprint arXiv:2312.15122*, 2023.
- [91] L. Rowe, R. Girgis, A. Gosselin, B. Carrez, F. Golemo, F. Heide, L. Paull, and C. Pal. Ctrl-sim: Reactive and controllable driving agents with offline reinforcement learning. In *8th Annual Conference on Robot Learning*, 2024.
- [92] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros. Image-to-image translation with conditional adversarial networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1125–1134, 2017.
- [93] K. Bousmalis, A. Irpan, P. Wohlhart, Y. Bai, M. Kelcey, M. Kalakrishnan, L. Downs, J. Ibarz, P. Pastor, K. Konolige, et al. Using simulation and domain adaptation to improve efficiency of deep robotic grasping. In *2018 IEEE international conference on robotics and automation (ICRA)*, pages 4243–4250. IEEE, 2018.
- [94] Z. Li, W. Yao, Z. Wang, X. Sun, J. Chen, N. Chang, M. Shen, J. Song, Z. Wu, S. Lan, et al. Ztrs: Zero-imitation end-to-end autonomous driving with trajectory scoring. *arXiv preprint arXiv:2510.24108*, 2025.
- [95] L. Liu, C. Jia, G. Yu, Z. Song, J. Li, F. Jia, P. Wu, X. Hao, and Y. Luo. Guideflow: Constraint-guided flow matching for planning in end-to-end autonomous driving. *arXiv preprint arXiv:2511.18729*, 2025.
- [96] H. Tian, T. Li, H. Liu, J. Yang, Y. Qiu, G. Li, J. Wang, Y. Gao, Z. Zhang, L. Wang, et al. Sim-scale: Learning to drive via real-world simulation at scale. *arXiv preprint arXiv:2511.23369*, 2025.
- [97] L. Nguyen, M. Fauth, B. Jaeger, D. Dauner, M. Igl, A. Geiger, and K. Chitta. Lead: Minimizing learner-expert asymmetry in end-to-end driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 39775–39785, 2026.
- [98] K. Chitta, D. Dauner, and A. Geiger. Sledge: Synthesizing driving environments with generative models and rule-based traffic. In *European Conference on Computer Vision*, pages 57–74. Springer, 2024.
- [99] L. Rowe, R. Girgis, A. Gosselin, L. Paull, C. Pal, and F. Heide. Scenario dreamer: Vectorized latent diffusion for generating driving simulation environments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17207–17218, 2025.
- [100] J. Suarez. PufferLib: Making reinforcement learning libraries and environments play nice. *arXiv preprint arXiv:2406.12905*, 2024.
- [101] P. Xiao, Z. Shao, S. Hao, Z. Zhang, X. Chai, J. Jiao, Z. Li, J. Wu, K. Sun, K. Jiang, et al. Pandaset: Advanced sensor suite dataset for autonomous driving. In *2021 IEEE international intelligent transportation systems conference (ITSC)*, pages 3095–3101. IEEE, 2021.
- [102] A. Geiger, P. Lenz, and R. Urtasun. Are we ready for autonomous driving? the kitti vision benchmark suite. In *2012 IEEE conference on computer vision and pattern recognition*, pages 3354–3361. IEEE, 2012.

- [103] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, et al. Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking. *Advances in Neural Information Processing Systems*, 37:28706–28719, 2024.

A Gigapixel Simulator

High-Throughput Batched Rendering. The Gigapixel simulator extends PufferDrive 2.0 [17], which is built on PufferLib 3.0 [100]. PufferLib 3.0 targets high-throughput RL training on *vectorized* observations, but does not natively support high-throughput rendering for training policies on pixel observations. Gigapixel closes this gap by integrating the GPU-accelerated Madrona batched perspective renderer [22], which supports both rasterized and ray-traced rendering. The Madrona renderer exposes an *entity-component-system* (ECS) abstraction: a scene is a set of *entities*, each described by a few typed *components* (e.g., position, orientation, color), and all rendering logic is expressed as data-parallel *systems* that read and update those components. Crucially, the same systems run across many independent *worlds* in lockstep on a single GPU, so thousands of environments render together in a single batched launch, which is the key to its high rendering throughput.

The driving scene maps onto this abstraction directly. We instantiate one Madrona world per ego agent, each rendering that agent’s ego-centric view. Within a world, every renderable element (surrounding vehicles, static obstacles, road segments, and traffic lights) shares a single entity archetype carrying only SE(3) transform and appearance components; what distinguishes a vehicle from a road segment or a traffic light is the primitive identifier stored in its geometry component. The ego camera and the scene’s directional light are themselves entities with their own small component sets. Because we utilize a deliberately simple abstract scene representation, a small set of geometric primitives suffices to realize the scene: vehicles and static objects use cuboids, road segments use planes, and traffic lights use spheres. Shading is minimal, using a single directional light per-agent with per-instance colour (set to blue for all agents and static objects in our experiments).

Rendering uses either a rasterized or a ray-traced backend. The rasterizer is faster, exploiting the simplicity of the primitives, while the ray tracer trades throughput for higher visual fidelity. Once the policy is paired with a compute-intensive architecture such as DrivoR [21], end-to-end inference and optimization dominate the per-step cost, so rendering is not the practical bottleneck; we therefore use the ray-traced backend as the default unless otherwise stated (see Figure 1 for a throughput analysis). The full path from simulator state to policy input stays on the GPU: rendered images are exposed directly as a batched GPU tensor to the learning framework, incurring no per-step CPU transfer. Image resolution and camera configuration (including the number of cameras) are configurable, trading visual fidelity against the per-step rendering budget available during self-play. Finally, the renderer scales to distributed training by assigning one GPU per rank. Because rendering is local to each rank and requires no inter-rank communication, throughput scales near-linearly with the number of GPUs.

Observations. Each controlled agent receives an ego-centric observation expressed in its own reference frame. The observation combines an ego-state vector, a high-level navigation command, and either rendered pixel observations (for pixel-based policies), or concatenated vectorized features (for vectorized policies).

The *ego state* comprises the signed longitudinal speed, longitudinal and lateral acceleration, the vehicle’s length and width, and a binary collision flag. Under the jerk dynamics model it additionally includes the current steering angle, and the longitudinal and lateral accelerations. All continuous quantities are normalized to $[-1, 1]$.

We summarize navigation intent with a discrete high-level command: LEFT, STRAIGHT, RIGHT, or UNKNOWN. This is the goal representation adopted by end-to-end AV benchmarks such as NAVSIM [28] and HUGSIM [27], in contrast to conditioning on a dense route or goal position. We derive the command geometrically from the agent’s logged trajectory: we snap the agent’s current position to the nearest point on its recorded path, advance a fixed look-ahead distance (20 m) along the path, and transform the resulting target into the egocentric frame. Its lateral offset determines the command, with offsets beyond ± 2 m mapping to LEFT or RIGHT and smaller offsets to STRAIGHT; UNKNOWN is emitted when no valid trajectory or snap point exists. The command is available both in self-play and at deployment, and does not expose the dense expert path to the policy.

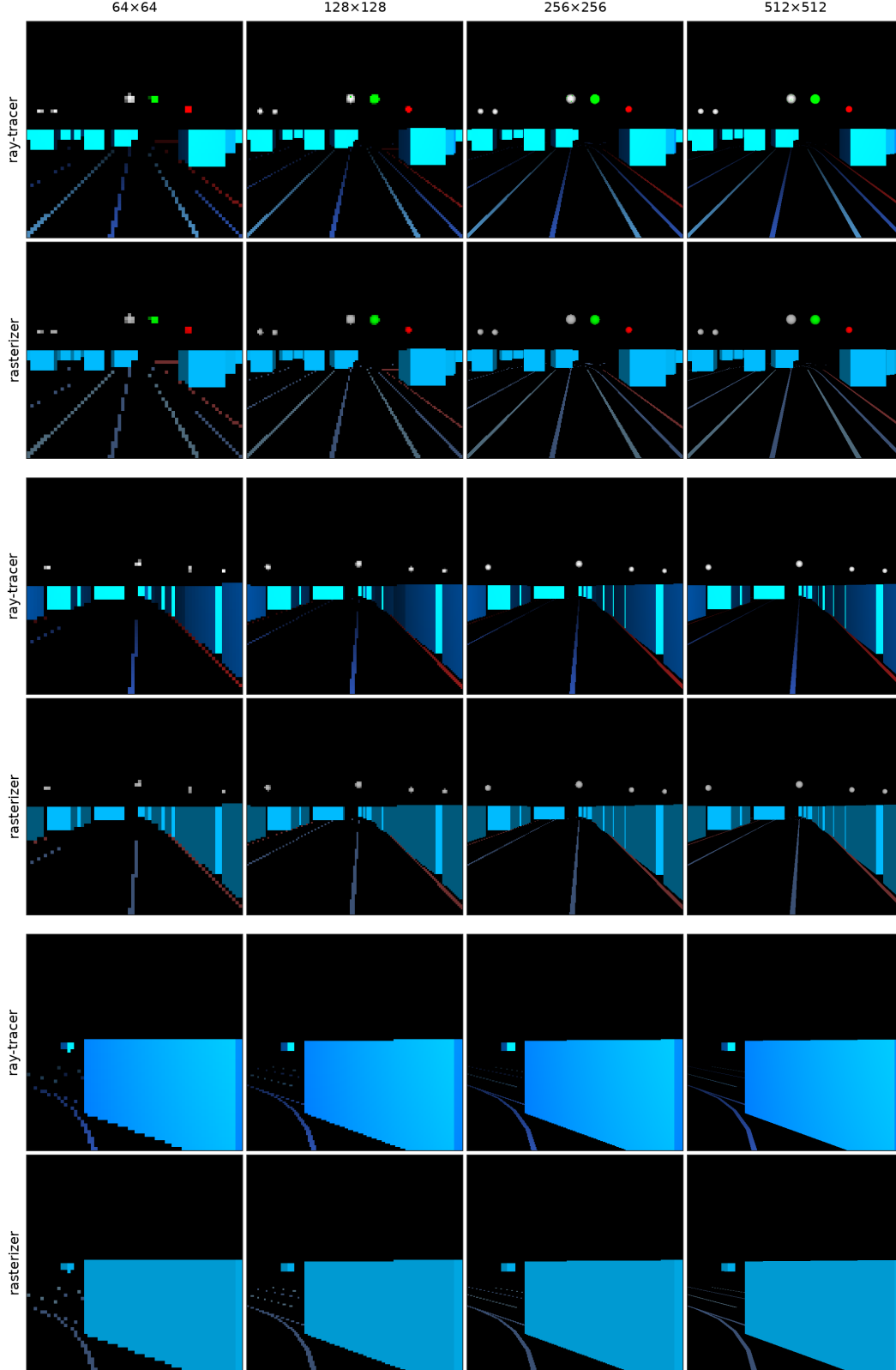


Figure 7: **Ray-traced vs. rasterized Gigapixel renderings.** We render the front-view of three scenes with both the Gigapixel ray-tracer and rasterizer across a range of resolutions. The rasterizer achieves higher throughput, but at the cost of simpler shading, as shown in the bottom two rows.

The vectorized features are: *surrounding agents* (the nearest K within 100 m), each contributing relative position, dimensions, relative heading as $(\cos, \sin)$, and relative signed speed; *road geometry*, encoded as polyline segments retrieved from a spatial hash around the agent, each carrying relative position, length, orientation as $(\cos, \sin)$, and a categorical type (e.g., lane, road edge, crosswalk); and *traffic controls*, giving the relative position and one-hot state (red/yellow/green/unknown) of the nearest signals. When reward conditioning is enabled, the per-agent reward-weight vector is appended to the ego state, allowing a single policy to enact different driving styles.

Action Space and Dynamics. The simulator supports two control regimes, depending on whether the policy emits low-level actions or a future trajectory.

Action-based policies. For policies that act directly, we adopt the same action space and dynamics as Gigaflow [16]. Each agent is controlled through a discrete set of twelve actions, formed as the Cartesian product of longitudinal jerk values $\{-15, -4, 0, 4\} \text{ m/s}^3$ and lateral jerk values $\{-4, 0, 4\} \text{ m/s}^3$, which drive a jerk-actuated kinematic bicycle model. At each step the selected jerks are integrated into longitudinal and lateral accelerations and then into velocity and steering, with accelerations clipped to g-force limits, speed clipped to the speed limit, and the steering angle and its rate bounded to keep trajectories kinematically feasible. The wheelbase is set to 60% of the vehicle length. We refer the reader to Gigaflow [16] for the full dynamics derivation.

Trajectory-based policies. Vision-based end-to-end planners such as DrivoR do not emit jerks directly; instead they predict a 4s future trajectory expressed as a sequence of future SE(2) waypoints $(\Delta x, \Delta y, \Delta \theta)$ in the ego-centric rear-axle frame, replanned at 2 Hz. To execute a planned trajectory in Gigapixel, we adapt the batched LQR controller from NAVSIM [28] and run it in a receding-horizon fashion. The predicted waypoints are interpolated to the simulation step ($\Delta t = 0.1 \text{ s}$) to form a dense reference of poses, velocities, and curvatures, which a pair of decoupled LQR controllers tracks: a longitudinal controller maps velocity error to a target acceleration, and a lateral controller maps lateral error, heading error, and steering angle to a steering rate, using a one-second lookahead. The resulting (acceleration, steering rate) commands are integrated through a kinematic bicycle model, and the propagated pose and velocity are written back to the simulator. For more details on the LQR controller, we refer readers to NAVSIM [28].

Reward Functions and Reward Conditioning. Following Gigaflow [16], the per-step reward is a weighted sum of interpretable terms covering safety, rule compliance, progress, and comfort. The safety terms penalize vehicle-to-vehicle collisions (scaled by impact speed) and leaving the drivable area. The rule-compliance terms penalize red-light violations, driving against the lane direction, exceeding the speed limit, and reversing. Progress is encouraged by a lane-aligned velocity reward and a terminal bonus for reaching the goal, with a small per-step time penalty to discourage stalling. Comfort is promoted by penalizing harsh acceleration and jerk, and a lane-centering term rewards staying near the lane centerline when aligned with it. The functional form of each term is given in Table 4.

To train a single policy that spans a range of driving styles, we randomize the reward coefficients per agent at reset and expose the sampled vector to the policy as an additional observation (*reward conditioning*). Each of the twelve coefficients is drawn independently and uniformly from the ranges in Table 4, normalized to $[-1, 1]$, and appended to the ego state. At inference time, the conditioning vector is fixed to a single nominal “careful-driver” setting (rightmost column of Table 4), selecting safe, rule-abiding behavior from the learned family of policies.

Other Details. The policy controls all vehicles by default, while pedestrians and cyclists follow their logged trajectories. Each episode runs for up to 20 s and terminates at the time limit or once all controlled agents have reached their goal. The goal position is set to the final xy coordinate in the human logs. Scenarios are extracted from nuPlan scenarios; each scenario provides road geometry (lanes, edges, crosswalks, stop signs), traffic-signal state sequences, and recorded trajectories for up to several hundred agents, from which agents’ initial states and high-level commands are derived.

Reward term	Definition	Training distribution	Inference
Collision	$-(\alpha_{\text{col}} + 0.1 v) \mathbb{1}_{\text{col}}$	$\alpha_{\text{col}} \sim \mathcal{U}(0, 3)$	3.0
Off-road	$-\alpha_{\text{off}} \mathbb{1}_{\text{off-road}}$	$\alpha_{\text{off}} \sim \mathcal{U}(0, 3)$	3.0
Comfort	$-\alpha_{\text{cmf}} (\mathbb{1}_{ a_{\text{lon}} >3} + \mathbb{1}_{ a_{\text{lat}} >3} + \mathbb{1}_{ \dot{a}_{\text{lon}} >5 \vee \dot{a}_{\text{lat}} >5})$	$\alpha_{\text{cmf}} \sim \mathcal{U}(0, 0.05)$	0.05
Lane alignment	$\alpha_{\text{aln}} \Delta t (\min(\cos \theta_f, 0) + \min(v \cos \theta_f, 0) + 0.0025 (1 - \frac{2\theta_f}{\pi}))$	$\alpha_{\text{aln}} \sim \mathcal{U}(2.5 \times 10^{-4}, 2.5 \times 10^{-2})$	0.025
Lane centering	$-\alpha_{\text{ctr}} \Delta t (\mathbb{1}_{\cos \theta_f > 0.5} x_f - 0.05 e^{0.5 - x_f })$	$\alpha_{\text{ctr}} \sim \mathcal{U}(2.5 \times 10^{-4}, 7.5 \times 10^{-3})$	3.8×10^{-3}
Velocity (progress)	$\alpha_{\text{vel}} \Delta t \max(\cos \theta_f, 0) \mathbb{1}_{ v >2.5}$	$\alpha_{\text{vel}} \sim \mathcal{U}(0, 5 \times 10^{-3})$	2.5×10^{-3}
Traffic-light	$-\alpha_{\text{tl}} \mathbb{1}_{\text{red-light violation}}$	$\alpha_{\text{tl}} \sim \mathcal{U}(0, 1)$	1.0
Time penalty	$-\alpha_t \Delta t \mathbb{1}_{ v >0 \vee a >0}$	$\alpha_t \sim \mathcal{U}(0, 5 \times 10^{-5})$	2.5×10^{-5}
Reverse	$-\alpha_{\text{rev}} \Delta t \mathbb{1}_{v<0}$	$\alpha_{\text{rev}} \sim \mathcal{U}(2.5 \times 10^{-4}, 7.5 \times 10^{-3})$	5×10^{-3}
Overspeed	$-\alpha_{\text{ovr}} \mathbb{1}_{ v >v_{\text{lim}}}$	$\alpha_{\text{ovr}} \sim \mathcal{U}(0, 1)$	0.0
Goal	$\mathbb{1}_{\ x-g\ <\delta_{\text{goal}}}$	$\delta_{\text{goal}} \sim \mathcal{U}(2, 12)$	5.0
Goal-speed tol.	$v_{\text{goal}} = v_{\text{max}}^{\text{human}} + \Delta v_{\text{tol}}$	$\Delta v_{\text{tol}} \sim \mathcal{U}(1, 5)$	5.0

Table 4: Reward terms, their functional form, and the per-agent randomization used for reward conditioning. Coefficients are positive magnitudes with the sign carried in the definition. θ_f is the heading misalignment with the lane, x_f the lateral offset from the lane center, v the speed, $a_{(\cdot)}$ and $\dot{a}_{(\cdot)}$ the accelerations and jerks, and Δt the simulation step. $v_{\text{max}}^{\text{human}}$ is the maximum speed along the agent’s human logged trajectory; the goal speed v_{goal} and the speed limit v_{lim} are both set to $v_{\text{max}}^{\text{human}} + \Delta v_{\text{tol}}$. The rightmost column gives the fixed conditioning used at inference.

On a collision, off-road departure, or red-light violation, the offending agent is halted in place (velocity zeroed) but continues to receive observations. When an agent reaches its goal, it respawns at its initial position in a parallel environment with no surrounding agents. Halting on infraction introduces more static obstacles during training, as rule-violating agents become static obstacles from the perspective of other agents. To make scenes more adversarial and less reactive, a configurable fraction of active agents can be demoted to *log replay*, following their recorded trajectories non-reactively rather than acting under the policy.

B Training Details

B.1 Vectorized Teacher Training

We train the teacher policy via self-play with decentralized PPO [86]. Reward weights are randomized per agent and supplied to the policy as conditioning inputs. We use the high-level discrete navigation command (LEFT/STRAIGHT/RIGHT/UNKNOWN) as the goal representation to reduce information asymmetry between the vectorized teacher and pixel-based student. Training takes 24 hours on 8 NVIDIA H200s. The remaining PPO and environment hyperparameters are listed in Table 5.

Policy architecture. The teacher is a compact ($\sim 2.7\text{M}$ -parameter) permutation-invariant network inspired by the GigafLOW architecture [16]. The flat observation is split back into its semantic groups: the ego state and the variable-length sets of surrounding agents, road-graph segments, and traffic light states. Each group is embedded by its own two-layer MLP encoder with layer normalization, and the three object sets are aggregated by element-wise max-pooling, yielding a fixed-size, permutation-invariant summary of the scene. The four group embeddings are concatenated and passed through a shared trunk of three GELU-activated, layer-normalized linear blocks, from which linear actor and value heads branch off. The actor produces the factorized (longitudinal, lateral) jerk action. The bulk of the parameters reside in this shared trunk.

Advantage prioritization. Driving experience is dominated by routine straight-line driving, while the maneuvers that matter most for driving competence (turns, yields, and near-collision recoveries) are comparatively rare. To focus optimization on these informative transitions, minibatch segments are sampled with probability proportional to the magnitude of their estimated advantage raised to a

Hyperparameter	Value
Algorithm	Decentralized PPO
Optimizer	Adam ($\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$)
Learning rate	5×10^{-4} (constant)
Discount γ	0.999
GAE λ	0.95
PPO clip ϵ	0.2
Value clip	0.2
Value loss coef.	0.5
Entropy coef.	0.005
Max gradient norm	1.0
Batch size	524,288
Minibatch size	32,768
Segment length	32
Update epochs	1
Advantage-priority α	0.85
Importance weight β_0	0.85
Priority clamp	5.0
Agents per map	512
Log-replay fraction	0.10
Episode length	67 steps ($\Delta t=0.3$ s, ≈ 20 s)
Training maps (nuPlan)	335,245
Total environment steps	2.5×10^{10}

Table 5: Key hyperparameters for vectorized teacher training. Reward randomization/conditioning ranges are listed separately in Table 4.

power α , rather than uniformly, upsampling high-magnitude advantage events. Advantage prioritization can be seen as a soft variant of advantage filtering proposed in Gigaflo_w [16].

B.2 Pixel-Based Student Training

We distill the vectorized teacher (Sec. B.1) into a pixel-based student via self-play DAGger in Gigapixel: the student observes only rendered images (plus a minimal ego state and high-level navigation command), while the teacher, retaining its privileged vectorized observations, provides on-policy supervision from states the student visits. Training takes 36 hours on 8 H200 GPUs, with training configurations listed in Table 6. We instantiate the student as a DrivoR model [21]. We train two variants: the full multimodal DrivoR planner, supervised with a winner-takes-all loss over its trajectory proposals, and a single-mode regression variant (DrivoR-Reg), supervised with an L_1 loss to the teacher trajectory. Relative to the original DrivoR, we make two modifications for compatibility with self-play training: (i) we augment the student’s ego state with vehicle length and width and the reward-conditioning vector so that it can control any agent in self-play, and (ii) we add a parallel *action head*, driven by a separate action query in the decoder, that predicts the discrete jerk action and steps the student when it controls actors other than the self-driving car (SDC).

Trajectory execution and supervision. Teacher trajectory targets τ are obtained by rolling out the teacher policy π_{teacher} in self-play mode for a horizon of $H = 4$ s at 10 Hz in a forked copy of the simulator. To execute a predicted trajectory in closed loop we use the LQR controller integrated in Gigapixel, but apply it only to the SDC: the controller’s kinematic-bicycle parameters are calibrated to nuPlan’s self-driving car geometry, so tracking buses or trucks would introduce large tracking errors. All other agents are instead stepped through Gigapixel’s native jerk dynamics using the student’s parallel action head, avoiding any vehicle-specific re-tuning of the controller. We match

Hyperparameter	Value
<i>Objective</i>	
Training mode	Self-play DAgger (trajectory + action)
Planning loss	Winner-takes-all (DrivoR) / L_1 (DrivoR-Reg)
Trajectory loss coef.	1.0
Action (KL) loss coef.	1.0
Cmd loss weights (L/F/R/U)	5/1/5/1
Teacher rollout horizon H	4 s
Teacher mix β	$1 \rightarrow 0$ linear over 125M, then 0
<i>Architecture</i>	
Vision backbone	DINOv2 ViT-S/14 (reg4), LoRA
Register tokens	16
Decoder depth / width	4 / 1024
Trajectory proposals	64 (DrivoR) / 1 (DrivoR-Reg)
Waypoints (horizon)	8 (2 Hz, 4 s)
<i>Optimization</i>	
Optimizer	AdamW (wd 0.01)
Learning rate	2×10^{-4} , step $\times 0.1$ at 100M
Batch / minibatch size	8,192 / 256
Max gradient norm	1.0
DAgger replay capacity	25,000
<i>Environment & rendering</i>	
Execution (SDC)	NAVSIM LQR, receding horizon control
Execution (non-SDC)	Student action head, native jerk dynamics
Cameras	4 (front, front-left, front-right, rear)
Resolution	$210 \times 126 \rightarrow 434 \times 252$ (final 25M)
Episode length	201 steps ($\Delta t = 0.1$ s, ≈ 20 s)
Training maps (nuPlan)	335,245
Total steps	125M + 25M

Table 6: Key hyperparameters for pixel-based student distillation. Reward randomization/conditioning ranges follow Table 4 and matched to the teacher during training.

the student’s reward conditioning vector $\mathbf{c}$ to the teacher, so that the student can be trained to imitate the same persona as the rolled out teacher.

Mixed-policy rollouts. Following standard DAgger practice [14], we use a mixed-policy rollout schedule: at each step, the executed action is drawn from the teacher with probability β and from the student with probability $1 - \beta$. We anneal β linearly from 1 to 0 over the first 125M steps and hold $\beta = 0$ thereafter, which limits chaotic self-play dynamics while the student is still converging and gradually shifts the visited-state distribution onto the student’s own rollouts.

Data curation. Because much of self-play is uninformative straight driving, and because imitating poor-quality trajectories is harmful, we curate the supervision in two ways. First, we reweight the trajectory loss by the high-level navigation command, upweighting the rarer left- and right-turn commands by $5\times$ relative to go-straight to counter the heavy class imbalance toward straight driving. Second, we filter the teacher rollouts that enter the replay buffer, discarding: trajectories whose H -second window contains any collision or off-road infraction; trajectories from agents that have already committed an infraction earlier in their current episode; and trajectories truncated before all waypoints are valid (e.g. by a goal respawn or episode end), which would otherwise create supervision asymmetry across waypoints.

Rendering and resolution schedule. Observations are produced by the Gigapixel ray-traced renderer, which provides front, front-left, front-right, and rear camera views. We train in two phases: the first 125M steps at 210×126 resolution, then a final 25M steps at 434×252 .

Scoring head. The scoring-based DrivoR variant ranks trajectory proposals with a learned scorer whose targets are produced by the nuPlan closed-loop simulator. Because the scoring head requires the nuPlan simulator for computing PDMS targets, we do not train the scoring head inside Gigapixel; instead, we first train all components except for the scoring head in Gigapixel, and then train only the scoring head in open-loop on the navtrain split using the DrivoR codebase, keeping all other modules frozen. We refer the reader to DrivoR [21] for the full scorer training details.

B.3 Sim-to-real Adaptation Training

The pixel student of Sec. B.2 is trained entirely on images from the synthetic Gigapixel renderer. To deploy it on real camera data, we add a final adaptation stage, transferring the policy from rendered observations to real images while preserving the behavior learned in self-play. The Gigapixel-trained student supplies the supervision targets, and we fine-tune the perception stack on real frames under the distillation objective detailed below. Following the DrivoR training configurations, we train the NAVSIM-v2 model for fewer epochs than the HUGSIM model and without the extra navval data: for NAVSIM-v2 we train for 10 epochs on navtrain (85k samples), and for HUGSIM we train for 20 epochs on navtrain+navval (103k samples). Both the teacher and the student operate at 434×252 resolution. We optimize with AdamW at a learning rate of 2×10^{-4} and a global batch size of 64 (16 per GPU across 4 GPUs), and set the perceptual feature loss coefficient to $\lambda = 1$. Training takes place on 4 A100 GPUs in roughly 12 hours.

Mode-aligned distillation objective. For the multimodal DrivoR planner, the teacher and student each output a set of $P = 64$ trajectory proposals, $\{\tau_k\}_{k=1}^P$ and $\{\hat{\tau}_k\}_{k=1}^P$, where each proposal $\tau_k \in \mathbb{R}^{H \times 3}$ is a sequence of H future SE(2) waypoints. During distillation, we supervise the student in a *mode-aligned* fashion: the student’s k -th proposal is regressed directly toward the teacher’s k -th proposal, rather than under a winner-takes-all assignment. Whereas DrivoR is normally trained against a single ground-truth target through a winner-takes-all loss $\min_k \|\hat{\tau}_k - \tau^*\|$, distillation instead applies dense, per-mode supervision against the teacher’s full proposal set:

$$\mathcal{L}_{\text{plan}} = \frac{1}{P} \sum_{k=1}^P \|\hat{\tau}_k - \tau_k\|_1. \quad (2)$$

The full distillation loss adds an L_2 feature-matching term on the decoder register tokens and the scoring head loss:

$$\mathcal{L} = \lambda \|E_{\text{per}}^{\text{real}} - E_{\text{per}}^{\text{sim}}\|_2^2 + \mathcal{L}_{\text{plan}} + \mathcal{L}_{\text{score}}, \quad (3)$$

where $E_{\text{per}}^{\text{sim}}, E_{\text{per}}^{\text{real}}$ are the teacher and student register tokens, and $\mathcal{L}_{\text{score}}$ is the scoring head loss (see DrivoR [21] for details). For the regression-based DrivoR-Reg variant, no scoring loss is applied and $P = 1$, which reduces $\mathcal{L}_{\text{plan}}$ to a single-mode L_1 regression toward the teacher trajectory, matching the planning loss used during self-play DAGger training in Gigapixel.

C Benchmark Details

We evaluate in three settings: our own Gigapixel simulator and two real-world driving benchmarks, HUGSIM [27] and NAVSIM-v2 [28]. Together these probe closed-loop driving on abstract rendered observations (Gigapixel), photorealistic closed-loop driving on reconstructed real scenes (HUGSIM), and pseudo-closed-loop planning on real logs (NAVSIM-v2).

Gigapixel. In Gigapixel we evaluate policies on a held-out set of 1,000 scenarios in closed loop, with all surrounding actors replaying their logged trajectories. This setting isolates closed-loop driving performance on Gigapixel’s abstract pixel observations, independent of any sim-to-real

adaptation. We report the *Gigapixel Driving Score*, $\max(0, \text{Completion Rate} - \text{Collision Rate} - \text{Off-road Rate})$.

HUGSIM. HUGSIM [27] is a closed-loop benchmark that reconstructs real-world logs in a 3D Gaussian-splatting simulator, enabling pixel-level closed-loop evaluation on real scenes. It aggregates four source datasets (nuScenes [29], Waymo [31], PandaSet [101], and KITTI [102]) and reports performance across four difficulty tiers (Easy, Medium, Hard, Extreme) that progressively increase the adversarial behavior of surrounding actors. The official metric is the HUGSIM Driving Score (HD-Score), which multiplies route-completion by a time-averaged, PDMS-style closed-loop score built from safety and comfort subscores (Sec. C.1). Following DrivoR [21], we evaluate on 345 scenarios.

NAVSIM-v2. NAVSIM-v2 [28] is a pseudo-closed-loop benchmark: planned trajectories are executed non-reactively against rule-based traffic. Its metric is the Extended Predictive Driver Model Score (EPDMS), which multiplicatively combines a Stage 1 score, computed on the real logged poses, with a Stage 2 score, computed on synthetically perturbed poses that probe robustness to off-distribution recovery states (Sec. C.1). We evaluate on the official leaderboard split, *navhard*.

C.1 Driving-Score Metrics

We provide high-level descriptions of the HUGSIM and NAVSIM-v2 driving scores and refer the reader to the respective papers for the full subscore definitions, weights, and formulas.

HUGSIM HD-Score. HUGSIM scores driving with a PDMS-style metric inspired by NAVSIM [103]. At each simulation step, the per-step score multiplies two safety subscores, no-collision and drivable-area compliance, by a weighted average of two additional subscores, time-to-collision and comfort. Because the safety terms enter multiplicatively, a collision or off-road event at a step drives that step’s score to zero. The final HD-Score averages the per-step score over the episode and multiplies it by a global route-completion score $RC \in [0, 1]$, the fraction of the intended route the policy completes. See HUGSIM [27] for the exact subscore definitions and weights.

NAVSIM-v2 EPDMS. NAVSIM-v2 extends the NAVSIM PDMS into the Extended PDMS (EPDMS). As in PDMS, the subscores split into multiplicative *penalty* terms that gate the overall score, no-collision (NC), drivable-area compliance (DAC), driving-direction compliance (DDC), and traffic-light compliance (TLC), and a *weighted-average* term over time-to-collision (TTC), ego progress (EP), lane keeping (LK), history comfort (HC), and extended comfort (EC). Any penalty violation zeroes the score, while the weighted-average term grades overall driving quality. EPDMS is computed in two stages whose scores are combined multiplicatively: Stage 1 evaluates the planner on the real logged poses, and Stage 2 evaluates it on synthetically perturbed poses (novel viewpoints offset from the logged trajectory), measuring robustness when recovering from off-distribution states. We refer the reader to NAVSIM-v2 [28] for the subscore weights and formulas.